★ Harden shop-api end to end, with security gates in CI

Level 11 capstone ~10 hours

1 Goal and outcome

You will take the sample stack (`shop-api`: a Node.js API behind Nginx, with PostgreSQL and Redis) from "works on my laptop" to "hard to attack and hard to ship with a known flaw". Concretely, by the end:

- The **host** that would run it has key-only SSH, a default-deny firewall and fail2ban (L11.01 §11.1).
- The **cloud definition** (Terraform, scanned but never applied) and an IAM policy follow least privilege (L11.02 §11.2, L11.03 §11.3).
- **No secret** lives in the image, the repository or an environment variable. Secrets are mounted as files (L11.04 §11.4) and a gitleaks hook and CI job stop leaks (L11.11).
- The **API** verifies JWTs strictly, enforces ownership, validates input and rate limits (L11.05 §11.5, L11.06 §11.6, L11.07 §11.7, L11.08 §11.8).
- The **containers** run as non-root, read-only, with all capabilities dropped, on a distroless base (L11.12).
- A **GitHub Actions pipeline** blocks merges on leaked secrets, vulnerable dependencies, vulnerable images and misconfigured IaC, and runs an OWASP ZAP baseline scan against the running stack (L11.09 §11.9, L11.10, L11.13).

Safety rule for the whole project: every attack, scan and "break it" step targets *your own* laptop, VM or repository only. Nothing here is aimed at a system you do not own. The Terraform in this project is scanned, never applied, so there is no cloud bill.

2 Architecture

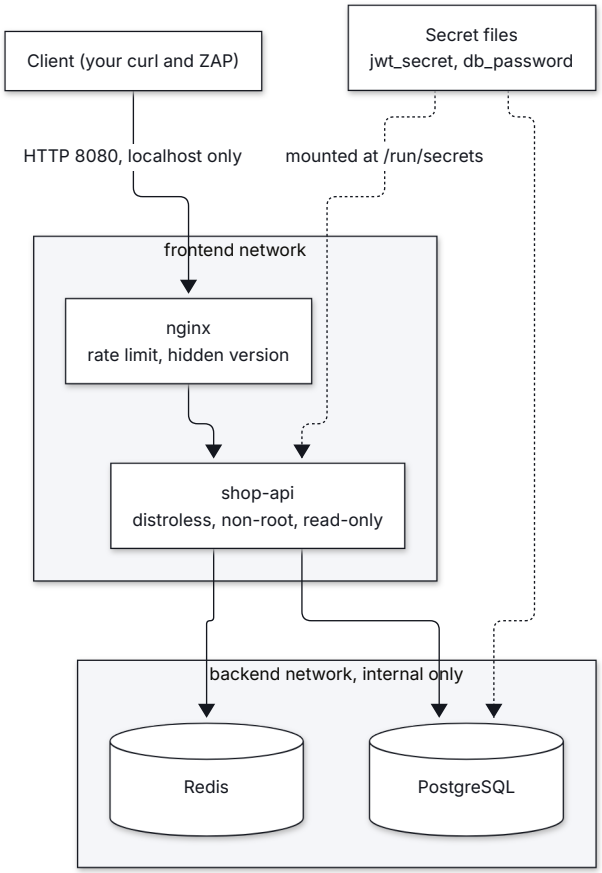


Figure 11-1

The `backend` network is `internal: true`, so the database and cache have no route to the internet and cannot be reached from the host. Only Nginx publishes a port, and only on `127.0.0.1`.

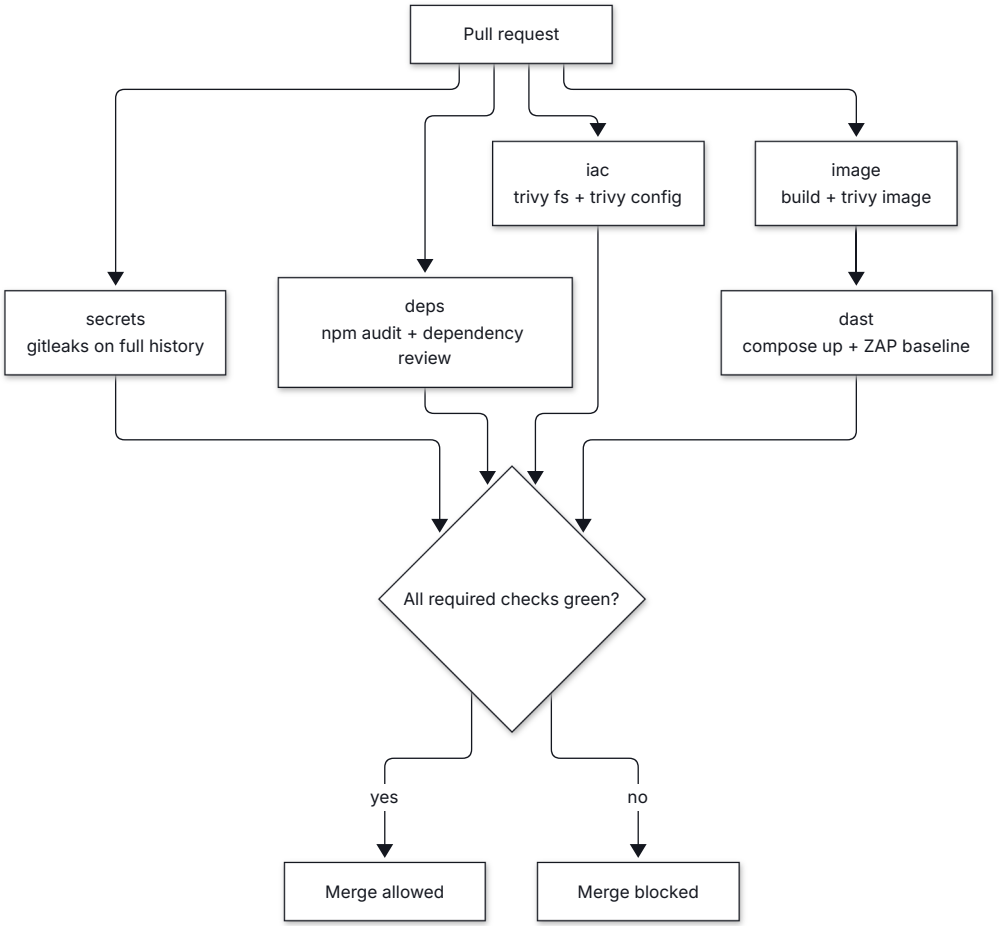


Figure 11-2

3 Prerequisites and cost

- A Linux or macOS machine with **Docker** and **Docker Compose v2**, **Node.js 22**, `git`, `curl`, `openssl`, `python3`. All free.
- A **GitHub account** and an empty repository you own (public repos get free Actions minutes and free secret scanning push protection).

- For milestone 1 only: a throwaway Linux VM you can reach through a console (Multipass, Vagrant, VirtualBox, or a small cloud VPS you will delete). **Never practise SSH or firewall changes on a server you cannot recover.**
- Optional: `terraform` (only `fmt` / `validate` are used). **No AWS account is needed**; the cloud files are scanned, not applied.
- Cost: **$0** locally. If you use a VPS for milestone 1, delete it afterwards (teardown step).
- Commands that call Docker, Trivy, gitleaks or ZAP print output *similar to* what is shown, not byte-identical; versions and vulnerability counts change daily.

4 Build it step by step

Milestone 0 — Create the baseline and threaten-model it

```bash
mkdir -p ~/shop-secure/{nginx,secrets,scripts,infra,.github/workflows,.zap} && cd ~/shop-
  secure
git init -b main
npm init -y >/dev/null
npm install express@4 helmet express-rate-limit jose
printf 'node_modules/\nsecrets/\n.env*\nzap-out/\n' > .gitignore
```

Write down a one-page threat model as `THREATS.md` before you build anything: list the assets (customer orders, the JWT signing key, the database), the entry points (Nginx port, SSH, CI), and for each of the OWASP Top 10 (L11.05 §11.5) categories one sentence on where it could hit `shop-api`. You will tick these off as the milestones close them.

Milestone 1 — Harden the host (on your throwaway VM)

Keep a second SSH session open while you do this. Create a deploy user with your public key first (see L11.01 §11.1 for the full walkthrough), then:

```bash
sudo tee /etc/ssh/sshd_config.d/10-hardening.conf >/dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
MaxAuthTries 3
AllowUsers deploy
EOF
sudo sshd -t && sudo systemctl reload ssh      # "sshd" on RHEL-family systems
sudo ufw default deny incoming && sudo ufw default allow outgoing
sudo ufw limit 22/tcp && sudo ufw allow 443/tcp
sudo ufw enable && sudo ufw status verbose
sudo apt-get install -y fail2ban unattended-upgrades && sudo systemctl enable --now
  fail2ban
sudo sshd -T | grep -Ei '^(permitrootlogin|passwordauthentication|allowusers)'
```

Expect `permitrootlogin no`, `passwordauthentication no`, `allowusers deploy`, and `Status: active` from ufw.

Milestone 2 — Cloud definition and least-privilege IAM (scan only)

`infra/main.tf` describes the security groups and bucket the stack would use on AWS (`us-east-1`). It is documentation-as-code that Trivy can judge:

```hcl
resource "aws_security_group" "alb" {
  name        = "shop-alb"
  description = "Public HTTPS to the load balancer"
  vpc_id      = var.vpc_id
  ingress {
    description = "HTTPS from the internet"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "app" {
  name        = "shop-app"
  description = "App tier, only reachable from the ALB"
  vpc_id      = var.vpc_id
  ingress {
    description     = "API port from ALB only"
    from_port       = 3000
```

```
    to_port         = 3000
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
}

resource "aws_security_group" "db" {
  name        = "shop-db"
  description = "PostgreSQL, only reachable from the app tier"
  vpc_id      = var.vpc_id
  ingress {
    description     = "PostgreSQL from app tier only"
    from_port       = 5432
    to_port         = 5432
    protocol        = "tcp"
    security_groups = [aws_security_group.app.id]
  }
}

variable "vpc_id" { type = string }
```

`infra/iam-shop-api.json` is the whole permission set the API task role needs (read its own uploads, nothing else):

```
JSON
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadOwnUploads",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::shop-example-uploads/uploads/*"
    }
  ]
}
```

Scan the folder (no AWS credentials are needed because Trivy only reads the files):

```
BASH
docker run --rm -v "$PWD":/work aquasec/trivy:latest config /work/infra --severity
  HIGH,CRITICAL
```

Expect no HIGH or CRITICAL findings for the files above. The `0.0.0.0/0` rule on the ALB is intentional (a public HTTPS entry point); if your Trivy version flags it anyway, record it as an accepted risk in `.trivyignore` with a reason and a review date (see Task T3). Findings change as Trivy's rules evolve, so read each one rather than assuming.

Milestone 3 — Secrets as files, and a gitleaks guard

```bash
chmod 700 secrets
openssl rand -base64 32 > secrets/jwt_secret
openssl rand -base64 24 | tr -d '/+=' > secrets/db_password
chmod 444 secrets/*          # readable by the non-root container user; the directory is
  700 so other host users cannot reach them
git check-ignore secrets/jwt_secret && echo "ignored by git: good"
```

Create `.gitleaks.toml` (the built-in rules stay on; this adds an allowlist for the test fixture you will use later) and a pre-commit hook as shown in L11.11:

```bash
cat > .gitleaks.toml <<'EOF'
title = "shop-secure gitleaks config"
[extend]
useDefault = true
[allowlist]
description = "Lab docs only"
paths = ['''THREATS\.md''']
EOF
cat > .pre-commit-config.yaml <<'EOF'
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: <LATEST_RELEASE_TAG>        # pin to a real release tag
    hooks:
      - id: gitleaks
EOF
pip install pre-commit && pre-commit install
```

Milestone 4 — The API: strict JWT verification, ownership, validation, limits

Save as `server.js`. It reads secrets from files, verifies a token's algorithm, issuer, audience and expiry, derives the order owner from the token (never the body), returns 404 for other people's orders, validates types and ranges, caps the body size, and rate limits. A dev-only token endpoint exists so you can test without an identity provider; it is off unless `ENABLE_DEV_TOKENS=true`.

```javascript
'use strict';
const fs = require('fs');
const express = require('express');
const helmet = require('helmet');
const { rateLimit } = require('express-rate-limit');
const { jwtVerify, SignJWT } = require('jose');

const readSecret = (name) => {
```

```
  const file = process.env[`${name}_FILE`];
  if (!file) throw new Error(`${name}_FILE is not set`);
  return fs.readFileSync(file, 'utf8').trim();
};
const key = new TextEncoder().encode(readSecret('JWT_SECRET'));

const app = express();
app.disable('x-powered-by');
app.use(helmet());
app.use(express.json({ limit: '10kb' }));
app.use(rateLimit({ windowMs: 60_000, limit: 100, standardHeaders: true, legacyHeaders:
  false }));

const orders = new Map();            // lab only: data lives in memory
let nextId = 1;

async function auth(req, res, next) {
  const h = req.get('authorization') || '';
  if (!h.startsWith('Bearer ')) return res.status(401).json({ error: 'missing token' });
  try {
    const { payload } = await jwtVerify(h.slice(7), key, {
      algorithms: ['HS256'], issuer: 'shop-auth', audience: 'shop-api',
    });
    req.user = { id: payload.sub, role: payload.role };
    next();
  } catch {
    res.status(401).json({ error: 'invalid token' });
  }
}

app.get('/health', (req, res) => res.json({ status: 'ok' }));
app.get('/products', (req, res) => res.json([{ id: 1, name: 'mug', price: 9.5 }]));

app.post('/orders', auth, (req, res) => {
  const { productId, qty } = req.body || {};
  if (!Number.isInteger(productId) || !Number.isInteger(qty) || qty < 1 || qty > 100) {
    return res.status(400).json({ error: 'productId and qty must be integers (qty 1-100)'
  });
  }
  const order = { id: nextId++, owner: req.user.id, productId, qty };
  orders.set(order.id, order);
  res.status(201).json(order);
});

app.get('/orders/:id', auth, (req, res) => {
  const order = orders.get(Number(req.params.id));
  if (!order || (order.owner !== req.user.id && req.user.role !== 'admin')) {
    return res.status(404).json({ error: 'not found' });
  }
  res.json(order);
});

if (process.env.ENABLE_DEV_TOKENS === 'true') {
```

```
  app.post('/dev/token', async (req, res) => {
    const token = await new SignJWT({ role: req.body?.role === 'admin' ? 'admin' : 'user'
  })
      .setProtectedHeader({ alg: 'HS256' }).setSubject(String(req.body?.sub || 'u1'))
      .setIssuer('shop-auth').setAudience('shop-api').setExpirationTime('15m').sign(key);
    res.json({ token });
  });
}

app.use((err, req, res, next) => res.status(400).json({ error: 'bad request' }));
app.listen(3000, '0.0.0.0', () => console.log('shop-api listening on 3000'));
```

Run it directly first (no Docker) with dev tokens on and prove the controls:

BASH

```
JWT_SECRET_FILE=$PWD/secrets/jwt_secret ENABLE_DEV_TOKENS=true node server.js &
TOKEN=$(curl -s -XPOST localhost:3000/dev/token -H 'content-type: application/json' -d
  '{"sub":"alice"}' | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')
curl -s -XPOST localhost:3000/orders -H "authorization: Bearer $TOKEN" -H 'content-type:
  application/json' -d '{"productId":1,"qty":2}'
# {"id":1,"owner":"alice","productId":1,"qty":2}
curl -s -XPOST localhost:3000/orders -H "authorization: Bearer $TOKEN" -H 'content-type:
  application/json' -d '{"productId":"1","qty":2}'
# {"error":"productId and qty must be integers (qty 1-100)"}
BOB=$(curl -s -XPOST localhost:3000/dev/token -H 'content-type: application/json' -d
  '{"sub":"bob"}' | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')
curl -s -w ' %{http_code}\n' localhost:3000/orders/1 -H "authorization: Bearer $BOB"     #
  {"error":"not found"} 404
kill %1
```

Save the checks you will reuse in CI as `scripts/security-smoke.sh`:

BASH

```
#!/usr/bin/env bash
# Usage: BASE=http://127.0.0.1:8080 scripts/security-smoke.sh   (own lab only)
set -u
BASE=${BASE:-http://127.0.0.1:8080}
fail=0
check() {   # check "<name>" "<expected>" "<actual>"
  if [ "$2" = "$3" ]; then echo "PASS $1"; else echo "FAIL $1 (expected $2, got $3)";
  fail=1; fi
}
code() { curl -s -o /dev/null -w '%{http_code}' "$@"; }

check "health is public"            200 "$(code "$BASE/health")"
check "orders without token"        401 "$(code -XPOST "$BASE/orders")"
check "forged alg=none token"       401 "$(code -XPOST "$BASE/orders" -H "authorization:
  Bearer eyJhbGciOiJub25lIn0.eyJzdWIiOiJhZG1pbiIsInJvbGUiOiJhZG1pbiJ9.")"
check "dev token endpoint is off"   404 "$(code -XPOST "$BASE/dev/token")"
```

```
check "no X-Powered-By header"      0   "$(curl -sI "$BASE/health" | grep -ci '^x-powered-
  by')"
check "nosniff header present"      1   "$(curl -sI "$BASE/health" | grep -ci '^x-content-
  type-options: nosniff')"
check "server version hidden"       0   "$(curl -sI "$BASE/health" | grep -Eci '^server:
  .*[0-9]+\.[0-9]+')"
exit $fail
```

`chmod +x scripts/security-smoke.sh`. Run against the app started *without* dev tokens; on a bare `node server.js` use `BASE=http://127.0.0.1:3000` (the last check then fails because there is no Nginx yet, which is expected until milestone 5).

Milestone 5 — Minimal, non-root, read-only containers

`Dockerfile`:

DOCKERFILE

```
FROM node:22-slim AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY server.js ./

FROM gcr.io/distroless/nodejs22-debian12:nonroot
WORKDIR /app
COPY --from=build /app /app
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD ["/nodejs/bin/node", "-e",
  "fetch('http://127.0.0.1:3000/health').then(r⇒process.exit(r.ok?
  0:1)).catch(()⇒process.exit(1))"]
CMD ["server.js"]
```

`nginx/default.conf`:

```nginx
limit_req_zone $binary_remote_addr zone=perip:10m rate=10r/s;

server {
    listen 8080;
    server_tokens off;
    client_max_body_size 16k;

    location / {
        limit_req zone=perip burst=20 nodelay;
        proxy_pass http://api:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

`compose.yaml`:

```yaml
services:
  nginx:
    image: nginx:1.27-alpine
    ports: ["127.0.0.1:8080:8080"]
    volumes: ["./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro"]
    read_only: true
    tmpfs: ["/var/cache/nginx", "/var/run", "/tmp"]
    cap_drop: [ALL]
    cap_add: [CHOWN, SETGID, SETUID]
    security_opt: ["no-new-privileges:true"]
    networks: [frontend]
    depends_on: [api]
  api:
    build: .
    image: shop-api:secure
    environment:
      JWT_SECRET_FILE: /run/secrets/jwt_secret
      DB_PASSWORD_FILE: /run/secrets/db_password
    secrets: [jwt_secret, db_password]
    read_only: true
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    pids_limit: 100
    mem_limit: 256m
    networks: [frontend, backend]
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
```

```
    volumes: ["dbdata:/var/lib/postgresql/data"]
    tmpfs: ["/var/run/postgresql"]
    cap_drop: [ALL]
    cap_add: [CHOWN, DAC_OVERRIDE, FOWNER, SETGID, SETUID]
    security_opt: ["no-new-privileges:true"]
    networks: [backend]
  cache:
    image: redis:7-alpine
    command: ["redis-server", "--save", "", "--appendonly", "no"]
    read_only: true
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    networks: [backend]
networks:
  frontend: {}
  backend:
    internal: true
volumes:
  dbdata: {}
secrets:
  jwt_secret: { file: ./secrets/jwt_secret }
  db_password: { file: ./secrets/db_password }
```

The lab API keeps orders in memory and does not open the database connection, but the database and cache still run so the network isolation, secret mounting and capability dropping are exercised on every service. Bring it up and check:

BASH

```
docker compose up -d --build
docker compose ps                                    # all services "running"; api shows
  (healthy) after ~30s
docker compose exec api id 2>&1 | head -1            # fails: distroless has no shell or id
  binary (that is the point)
docker inspect "$(docker compose ps -q api)" --format '{{.Config.User}}
  {{.HostConfig.ReadonlyRootfs}} {{.HostConfig.CapDrop}}'
# similar to: 65532 true [ALL]
curl -s -i http://127.0.0.1:8080/health | head -3    # HTTP/1.1 200 OK, "Server: nginx"
  without a version
nc -zv 127.0.0.1 5432 2>&1 | tail -1                 # connection refused: the database is
  not published
scripts/security-smoke.sh                            # 7 PASS lines, exit 0
```

If a service restarts in a loop, read `docker compose logs <service>`: a read-only filesystem or a dropped capability is the usual cause, and the fix is a specific `tmpfs` or one named `cap_add`, never `privileged: true`.

Milestone 6 — Wire the scans into CI

`.github/dependabot.yml`:

```yaml
version: 2
updates:
  - package-ecosystem: npm
    directory: "/"
    schedule: { interval: weekly }
  - package-ecosystem: docker
    directory: "/"
    schedule: { interval: weekly }
  - package-ecosystem: github-actions
    directory: "/"
    schedule: { interval: weekly }
```

`.zap/rules.conf` (rule id, action, comment; tab-separated). Start by enforcing the header rules you fixed with Helmet and leave the rest as warnings:

```
10020     FAIL    (Missing Anti-clickjacking Header)
10021     FAIL    (X-Content-Type-Options Header Missing)
10038     WARN    (Content Security Policy Header Not Set)
10049     IGNORE  (Non-Storable Content, expected for an API)
```

`.github/workflows/security.yml`:

```yaml
name: security
on:
  pull_request:
  push:
    branches: [main]
  schedule:
    - cron: "17 4 * * 1"          # weekly rescan: new CVEs appear for code that did not
  change
permissions:
  contents: read

jobs:
  secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }       # full history, a shallow clone hides old leaks
      - name: gitleaks
        run: |
          docker run --rm -v "$PWD":/repo zricethezav/gitleaks:<PINNED_VERSION> \
            git /repo -c /repo/.gitleaks.toml --redact -v
```

```
  deps:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm audit --audit-level=high --omit=dev
      - run: npm audit signatures
      - uses: actions/dependency-review-action@v4
        if: github.event_name == 'pull_request'
        with: { fail-on-severity: high }

  iac:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Trivy filesystem scan (vulnerabilities and secrets)
        uses: aquasecurity/trivy-action@0.28.0        # pin to the full commit SHA of the
  release you reviewed
        with:
          scan-type: fs
          scanners: vuln,secret
          severity: HIGH,CRITICAL
          ignore-unfixed: true
          exit-code: "1"
      - name: Trivy config scan (Dockerfile, compose, Terraform)
        uses: aquasecurity/trivy-action@0.28.0
        with:
          scan-type: config
          severity: HIGH,CRITICAL
          exit-code: "1"

  image:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t shop-api:ci .
      - name: Trivy image scan
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: shop-api:ci
          severity: HIGH,CRITICAL
          ignore-unfixed: true
          exit-code: "1"

  dast:
    needs: image
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Create throwaway CI secrets
```

```
        run: |
          mkdir -p secrets && chmod 700 secrets
          openssl rand -base64 32 > secrets/jwt_secret
          openssl rand -base64 24 | tr -d '/+=' > secrets/db_password
          chmod 444 secrets/*
      - name: Start the stack
        run: |
          docker compose up -d --build
          for i in $(seq 1 30); do curl -fsS http://127.0.0.1:8080/health && break; sleep
2; done
      - name: Security smoke tests
        run: scripts/security-smoke.sh
      - name: ZAP baseline scan (own CI stack only)
        run: |
          mkdir -p zap-out && cp .zap/rules.conf zap-out/ && chmod -R 777 zap-out
          docker run --rm --network host -v "$PWD/zap-out":/zap/wrk:rw -t
ghcr.io/zaproxy/zaproxy:stable \
            zap-baseline.py -t http://127.0.0.1:8080 -c rules.conf -r report.html -J
report.json || [ $? -eq 2 ]
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: zap-report, path: zap-out/ }
      - if: always()
        run: docker compose down -v
```

How the gates behave: Trivy, gitleaks and `npm audit` exit non-zero on findings and fail the job. ZAP exits `1` when a rule set to `FAIL` fires, `2` for warnings only, and `0` for clean, so the `|| [ $? -eq 2 ]` tolerates warnings while still failing on `FAIL` rules. The rules file is copied into the mounted `/zap/wrk` folder because ZAP's scripts read config files relative to it.

Finally, make the checks *required*: GitHub repository Settings, Branches, add a rule for `main`, require the `secrets`, `deps`, `iac`, `image` and `dast` status checks and a pull request before merging (L11.13 and L11.09 §11.9 explain the tuning). Also enable Dependabot alerts and secret scanning push protection under Settings, Code security.

Milestone 7 — Break it on purpose, then prove each gate

For each control, introduce the flaw on a branch, push, and confirm the pull request is blocked; revert before merging. Task T8 lists the exact flaws.

5 Tasks

- [ ] **T1 — Build the baseline and write the threat model** *(≈45 min, beginner)* — Milestone 0; topic L11.05 §11.5
  - **Goal:** a repository with the app skeleton and a `THREATS.md` that names assets, entry points and one OWASP risk per category.

- **Steps:**
  1. Run the Milestone 0 commands.
  2. Write `THREATS.md` with the three lists and a table: OWASP category, where it could apply to `shop-api`, which milestone mitigates it.
  3. Commit.
- **Done when:** `git log --oneline` shows the commit and all ten OWASP categories have a row with a milestone number (or "accepted" with a reason).

☐ **T2 — Harden a VM and prove you cannot get in the old way** *(≈60 min, intermediate)* — Milestone 1; topic L11.01 §11.1

- **Goal:** key-only SSH, default-deny firewall, fail2ban active on a throwaway VM you own.
- **Steps:**
  1. Apply the Milestone 1 configuration with a second session open.
  2. From the host run `ssh -o PreferredAuthentications=password deploy@<VM_IP>` and `ssh root@<VM_IP>`.
  3. Run `sudo ufw status verbose` and `sudo fail2ban-client status sshd`.
- **Done when:** both login attempts are refused (`Permission denied (publickey)`), ufw shows default deny incoming, and fail2ban lists the `sshd` jail as active.

☐ **T3 — Scan the cloud definition and write a least-privilege policy** *(≈60 min, intermediate)* — Milestone 2; topics L11.02 §11.2, L11.03 §11.3

- **Goal:** `trivy config` is clean for `infra/`, and every permission in the IAM policy is justified.
- **Steps:**
  1. Create `infra/main.tf` and `infra/iam-shop-api.json` and run the Trivy config scan.
  2. Change the `db` ingress to `cidr_blocks = ["0.0.0.0/0"]` and rescan; read the finding.
  3. Revert it. Write one sentence per IAM statement saying which API code path needs it. If you accept any remaining finding, add it to `.trivyignore` with a comment and a review date.
- **Done when:** the scan fails with the open database rule and passes after the revert, and the IAM policy has no `*` actions or resources.

☐ **T4 — Keep secrets out of Git and prove the guard works** *(≈45 min, intermediate)* — Milestone 3; topics L11.04 §11.4, L11.11

- **Goal:** a leaked fake key is stopped at commit time and in CI.

- **Steps:**
  1. Install the hook (`pre-commit install`).
  2. Add a line `AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE` and a matching fake secret to a file named `config.js` and try to commit it.
  3. Bypass the hook with `git commit --no-verify` on a branch, push, and watch the `secrets` CI job (once Milestone 6 exists) fail.
  4. Remove the file from the branch history (`git reset` before pushing, or rewrite the branch), never just delete it in a follow-up commit.
- **Done when:** the hook blocks step 2 with a gitleaks finding, CI fails on step 3, and `gitleaks git` on the cleaned branch reports no leaks.

☐ **T5 — Attack your own API and prove the controls** *(≈90 min, intermediate)* — Milestone 4; topics L11.05 §11.5, L11.06 §11.6, L11.07 §11.7, L11.08 §11.8

- **Goal:** demonstrate four attacks against **your local** `shop-api` and show each is refused.
- **Steps:**
  1. Start the app with dev tokens on. As `alice`, create an order; as `bob`, request `/orders/1` (broken object-level authorization).
  2. Send a token with `alg: none` (the one in the smoke script) and a token signed with a wrong key.
  3. POST `{"productId":"1; DROP TABLE","qty":-5}` and an 11 KB body.
  4. Send 120 requests in one minute (`for i in $(seq 120); do curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/products; done | sort | uniq -c`).
- **Done when:** the results are, in order, `404`, `401` twice, `400` and a `400` / `413`-style rejection, and some of the 120 requests return `429`. Add each as a new `check` line in the smoke script.

☐ **T6 — Run the stack hardened and verify each setting from outside** *(≈75 min, intermediate)* — Milestone 5; topics L11.12, L11.09 §11.9

- **Goal:** the running containers match the claims in `compose.yaml`.
- **Steps:**
  1. `docker compose up -d --build` and run the Milestone 5 checks.
  2. Scan the image: `docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest image --severity HIGH,CRITICAL shop-api:secure`.
  3. Show the container cannot be modified or entered: `docker compose exec api sh` fails (no shell in distroless), and `docker inspect` reports `ReadonlyRootfs true`.
  4. Confirm the database port is unreachable from the host and the API can reach it only over the `backend` network.

- **Done when:** `docker inspect` shows a non-root user, read-only root filesystem and `CapDrop [ALL]`; the smoke script is all PASS; and the image scan is clean or every remaining finding is documented.

☐ **T7 — Ship the security pipeline and make it required** *(≈90 min, advanced)* — Milestone 6; topics L11.09 §11.9, L11.10, L11.11, L11.13

- **Goal:** all five jobs run on every pull request and `main` is protected by them.
- **Steps:**
  1. Add the workflow, Dependabot file and ZAP rules; push to a feature branch and open a pull request.
  2. Fix whatever the first run finds (pin `<PINNED_VERSION>`, adjust the ZAP rules file path, add a `.trivyignore` entry with a reason for anything you accept).
  3. Enable branch protection requiring the five checks.
  4. Download the `zap-report` artifact and read the alerts.
- **Done when:** a pull request shows five green checks, the merge button is disabled while any of them is red, and the ZAP report is attached to the run.

☐ **T8 — Break each gate on purpose (own repository only)** *(≈90 min, advanced)* — Milestone 7; topics L11.09 §11.9, L11.10, L11.11, L11.13

- **Goal:** see every gate go red for the right reason, then green again.
- **Steps:** on separate branches, one flaw each:
  1. Commit the fake `AKIAIOSFODNN7EXAMPLE` key (secrets job fails).
  2. Pin a known-vulnerable old version, e.g. `npm install lodash@4.17.15` (deps job fails; use any dependency with a published advisory).
  3. Change the base image to `node:16` (image job fails) or open the database rule in Terraform (iac job fails).
  4. Remove `app.use(helmet())` (smoke test fails on `nosniff`, and the ZAP `FAIL` rules trip).
- **Done when:** you have four red pull requests, each failing in the job you predicted before pushing, and each turns green after you revert the flaw.

☐ **T9 — Prove it: explain and re-create the hardening from memory** *(≈2 hours, advanced)*

- **Goal:** you can defend every control and rebuild the pipeline without the guide.
- **Steps:**
  1. In a fresh directory, from memory, write a `compose.yaml` for a non-root, read-only API with a secret file and an internal-only database, and a workflow with at least gitleaks, `npm audit` and Trivy.

2. Present (aloud or in a short write-up) the path of one request through every control, naming the OWASP category each control addresses.
3. Answer: what would still get through this pipeline (hint: business logic flaws, zero-days, compromised build tooling)? List three controls from other levels that cover the gap, for example Kubernetes RBAC §8.18 or policy as code §18.7.

- **Done when:** your rebuilt stack passes `scripts/security-smoke.sh` and your write-up names at least five controls and at least three residual risks.

6 Verification checklist

- [ ] `sshd -T` shows root login and password authentication disabled; firewall default is deny incoming (on the VM).
- [ ] `trivy config infra/` has no unexplained HIGH or CRITICAL findings; the IAM policy contains no wildcard actions.
- [ ] `git check-ignore secrets/jwt_secret` prints the path; `gitleaks git` on the full history reports no leaks; the pre-commit hook blocks a fake key.
- [ ] `scripts/security-smoke.sh` prints only PASS and exits `0`.
- [ ] `docker inspect` on `api` shows a non-root user, `ReadonlyRootfs true`, and `CapDrop [ALL]`; no container is `privileged`.
- [ ] The database port (5432) and Redis port (6379) are not reachable from the host.
- [ ] Trivy image scan of `shop-api:secure` has no fixable HIGH or CRITICAL vulnerabilities.
- [ ] The five CI jobs pass on a clean pull request, and each fails when its flaw is introduced (T8).
- [ ] Branch protection requires the five checks on `main`; Dependabot and secret scanning push protection are enabled.
- [ ] `THREATS.md` has a mitigation or an accepted-risk reason for every OWASP Top 10 category.

7 Teardown

```bash
cd ~/shop-secure
docker compose down -v --rmi local          # stops the stack, removes the database volume
  and the locally built image
docker rmi aquasec/trivy:latest ghcr.io/zaproxy/zaproxy:stable 2>/dev/null
rm -rf zap-out secrets                      # the lab secrets are throwaway
pre-commit uninstall
cd ~ && rm -rf ~/shop-secure                # only when you are completely finished
```

Delete the throwaway VM or VPS you used in milestone 1 (for example `multipass delete --purge <vm>` or the provider's console), the test branches, and the GitHub repository if it was only for this exercise. If you generated any real credentials anywhere while experimenting, rotate them now. No AWS resources were created, so there is nothing to bill.

8 Stretch goals and what to learn next

- Replace the dev token endpoint with a real identity provider: run Keycloak as in L11.07 §11.7 and verify tokens against its JWKS endpoint with RS256 or ES256.
- Move the secret from files to HashiCorp Vault or AWS Secrets Manager with short-lived credentials (L11.04 §11.4).
- Produce an SBOM in CI (`trivy image --format cyclonedx`) and sign the image with cosign; fail deploys for unsigned images.
- Add a ZAP API scan from an OpenAPI file, and a nightly authenticated full scan against staging only.
- Add `trivy config` rules as custom policy, then enforce the same rules at cluster admission (policy as code §18.7).
- Next level: Level 12 — Advanced Networking §12.1 (TLS, mTLS and rate limiting in depth), and for the cluster-side version of these controls see Pod Security §16.3 and Kubernetes RBAC §8.18.